

# Relatório Projeto de POO

## Jogo de Estratégia



Fábio Cruz - 2025179383

Tiago Silva – 2024149084

Docente: Patrícia Alexandra Pires Macedo

Turma:PL5

# Índice

|                                     |    |
|-------------------------------------|----|
| Relatório Projeto de POO .....      | 1  |
| Apresentação do Projeto .....       | 3  |
| Arquitetura da Aplicação .....      | 8  |
| Modelo de Domínio .....             | 8  |
| Controlo da Aplicação .....         | 8  |
| Interface Gráfica .....             | 9  |
| Funcionalidades Implementadas ..... | 9  |
| Aplicação dos Conceitos de POO..... | 10 |
| Encapsulamento.....                 | 10 |
| Herança .....                       | 10 |
| Polimorfismo.....                   | 10 |
| Testes Unitários .....              | 11 |
| Dificuldades Estruturas .....       | 12 |
| Conclusão .....                     | 13 |
| Captura de Ecrã .....               | 13 |

## Apresentação do Projeto

O nosso projeto desenvolvido no âmbito da cadeira de POO consiste num jogo de estratégia por turnos, implementado em Java e com interface gráfica em Java FX. O jogo é composto por dois jogadores, que competem entre si ao decorrer de vários dias com o objetivo de alcançar a maior pontuação possível.

Cada Jogador tem um conjunto de recursos que são utilizados para realizar as ações do jogo, como construir ou melhorar estruturas.

O jogo decorre em um mapa em formato de grelha, em que os jogadores podem construir diversos tipos de estruturas. Cada estrutura tem características próprias, como tipo de produção e a quantidade produzida.

A interação com o jogador é feita através da interface gráfica desenvolvida em Java FX. O jogador pode iniciar um novo jogo, carregar um jogo guardado, visualizar o mapa e muito mais.

O jogo termina quando o último dia é alcançado. No final, é comparada a pontuação dos dois jogadores e é determinado o vencedor. Em caso de igualdade, não é atribuído um vencedor.

## Modelação do Domínio

Durante a realização do projeto, foram criadas várias entidades que representam os elementos essenciais da aplicação e a forma como interagem entre si.

A entidade Player representa cada jogador. Cada jogador possui um nome, uma pontuação e um inventário. O inventário é representado através de um HashMap, com os recursos associados às respetivas quantidades.

A entidade WorldMap representa o mapa do jogo. Este mapa é composto por uma matriz de estruturas. Cada posição pode estar vazia ou ocupada por uma estrutura.

A entidade Structures representa a classe abstrata para todas as estruturas do jogo. Esta classe contém os atributos e os comportamentos comuns a todas as estruturas, como o nível, o custo de manutenção, o lucro, o dono da estrutura e os métodos para gerar e consumir recursos e melhorar as estruturas.

As classes Forest, Mine, Ranch e City representam estruturas concretas. Cada uma destas classes herda de Structures e implementa o comportamento específico da respetiva estrutura. Por exemplo, a Forest produz madeira e a Mine produz pedra.

A classe GameSession contém a lógica principal das ações do jogo. Esta classe controla o estado global do jogo, incluindo o mapa, os jogadores, o dia atual e os eventos.

A classe GameEngine contém a lógica principal das ações do jogo. Esta classe permite criar estruturas, melhorar estruturas, procurar recursos e obter informações de

estruturas existentes no mapa.

A classe Events representa o sistema de eventos aleatórios. Estes eventos alteram temporariamente as regras do jogo, como por exemplo, a quantidade de pontuação que se obtém ou a quantidade de recursos que as estruturas produzem.

A classe FileHandler é responsável pela gravação e carregamento do jogo. Esta classe permite guardar e reconstruir o estado atual do jogo.

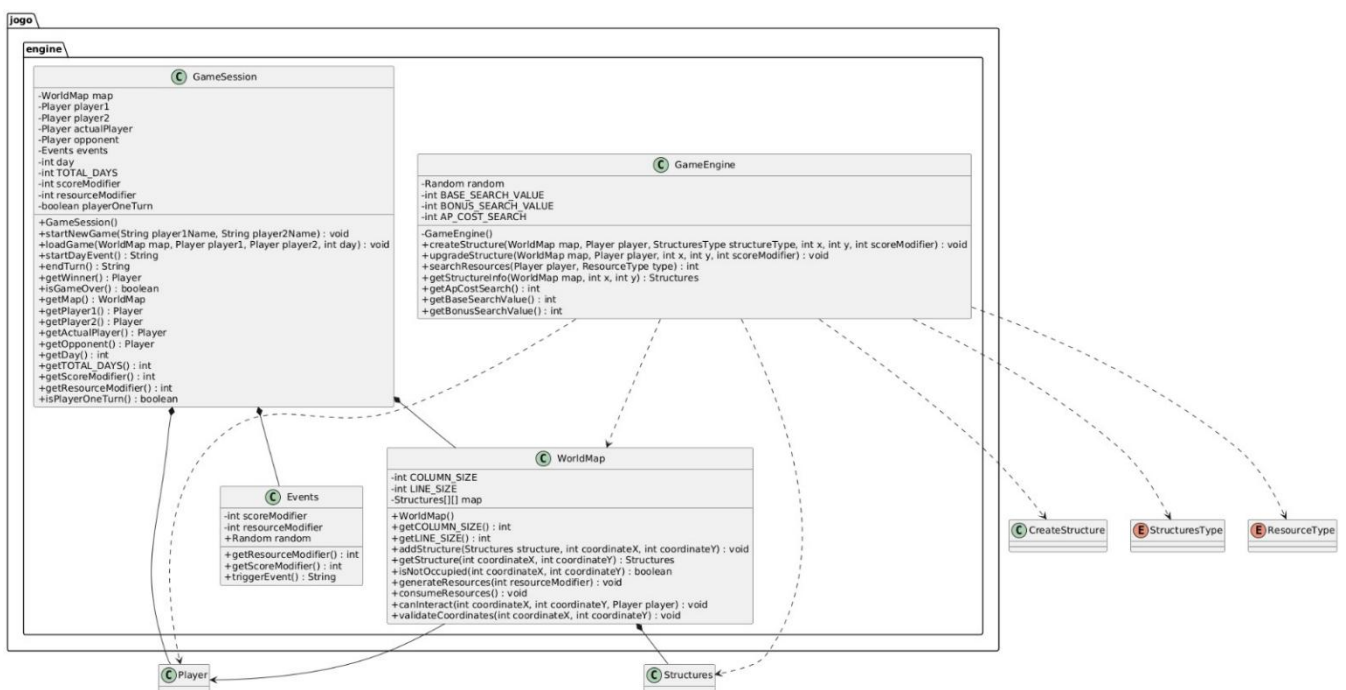
Para além disso, também foram criadas exceções personalizadas para representar situações de erro específicas do jogo, como a falta de recursos ou coordenadas inválidas.

## Diagrama de Classes UML

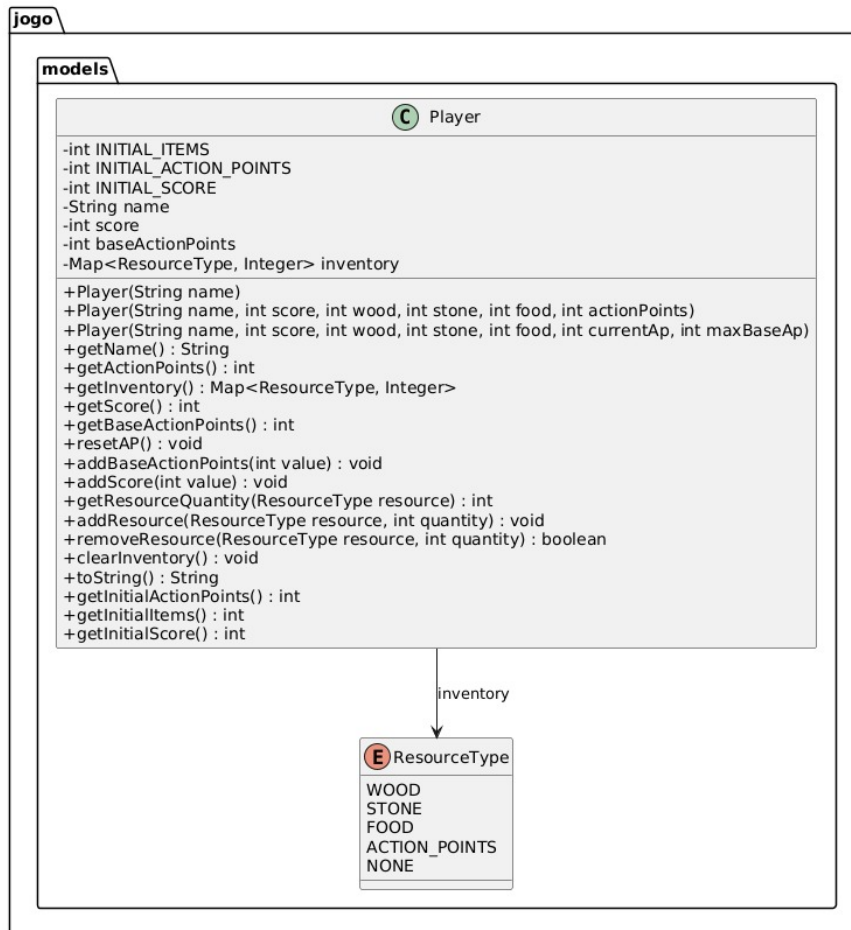
O diagrama de classes UML do jogo foi organizado por packages, para que seja mais fácil e intuitivo de compreender a estrutura do projeto. Esta divisão demonstra a separação do domínio do jogo, do motor da aplicação, das estruturas, do sistema de exceções e a interface gráfica

Foram elaborados diagramas UML para estes packages:

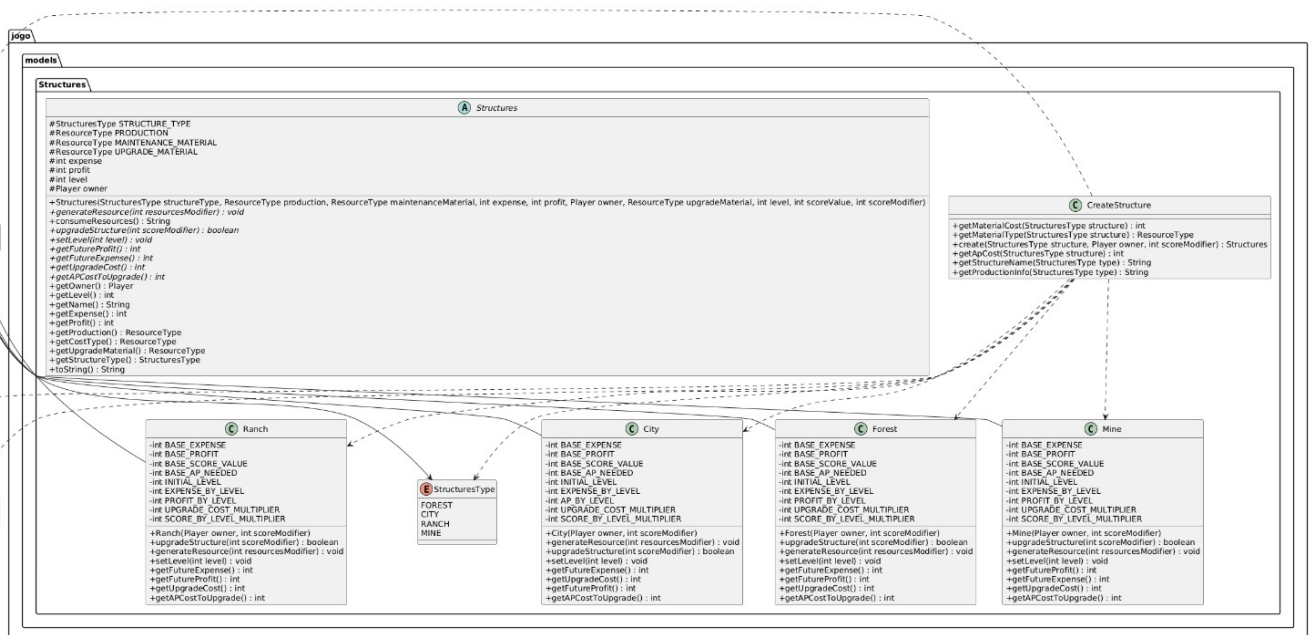
- jogo.engine
- jogo.models
- jogo.models.structures
- jogo.io
- jogo.exceptions
- jogo
- jogo.screens



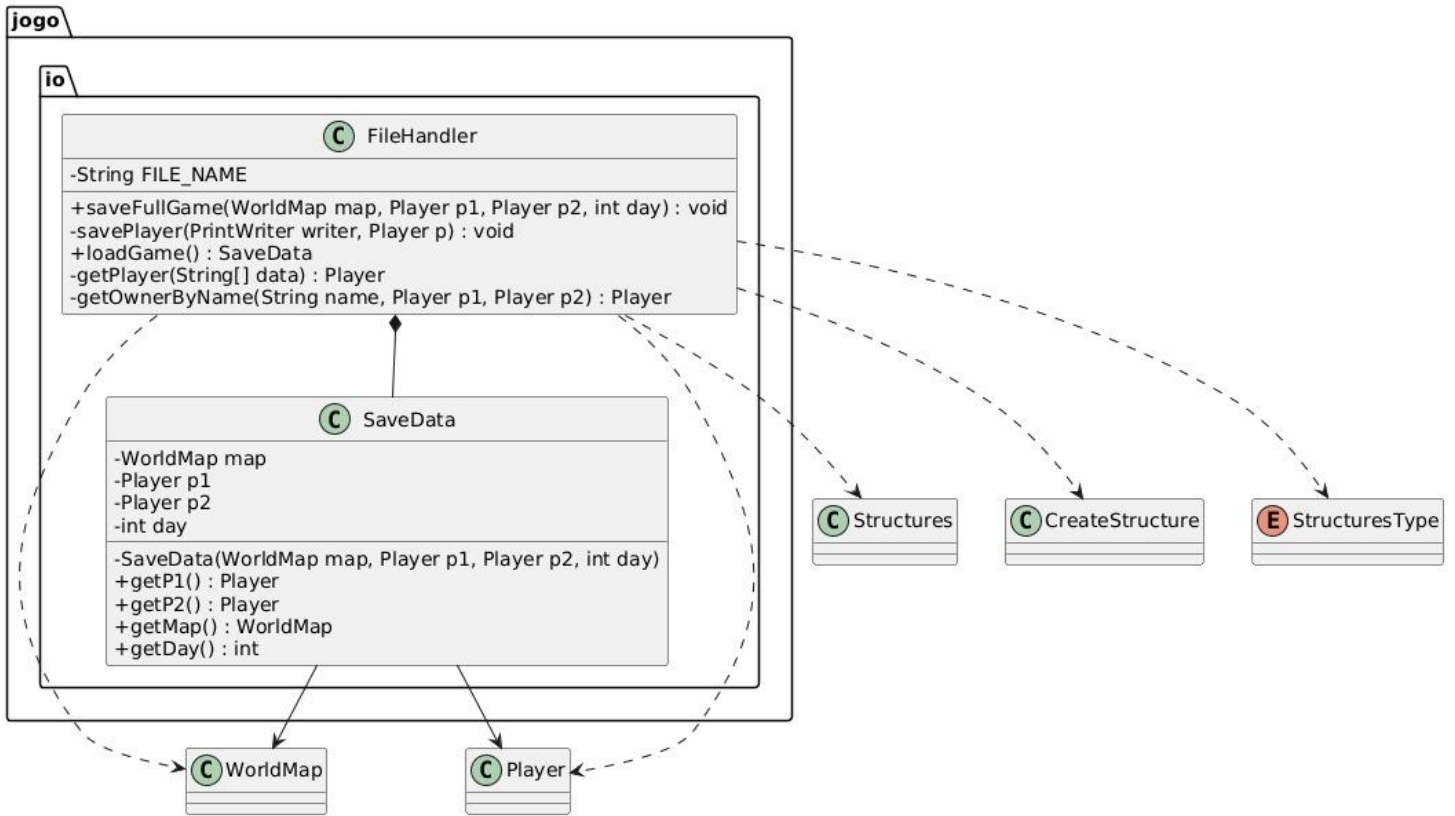
UML de jogo.engine



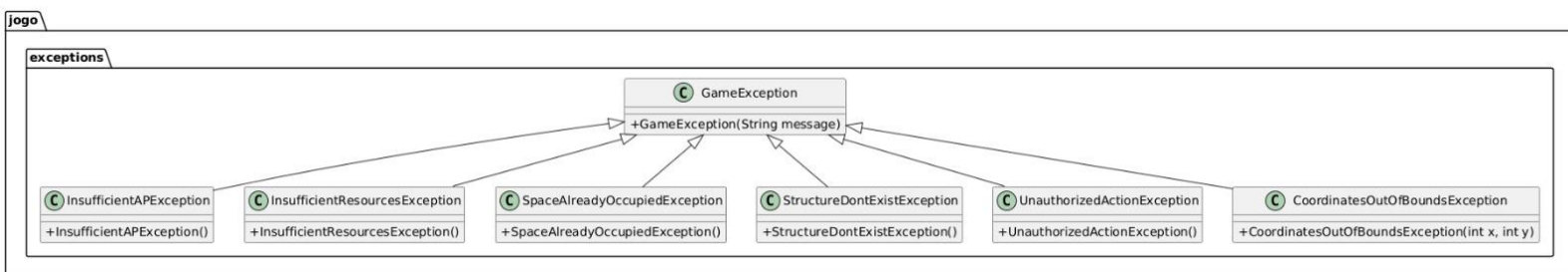
UML de jogo.models



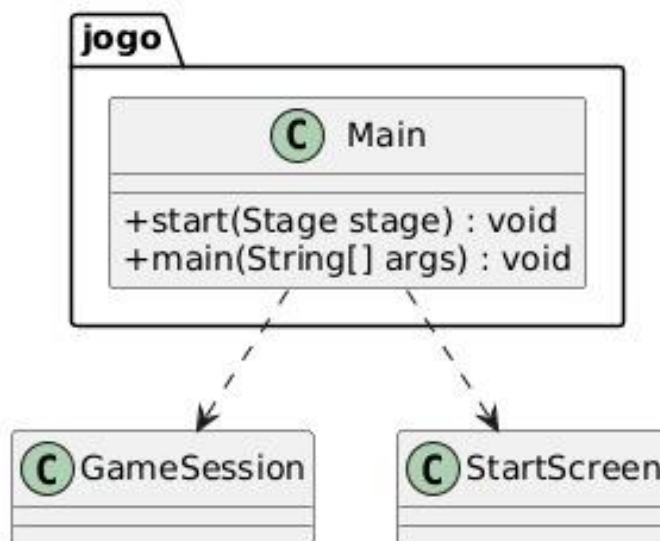
UML de jogos.models.structure



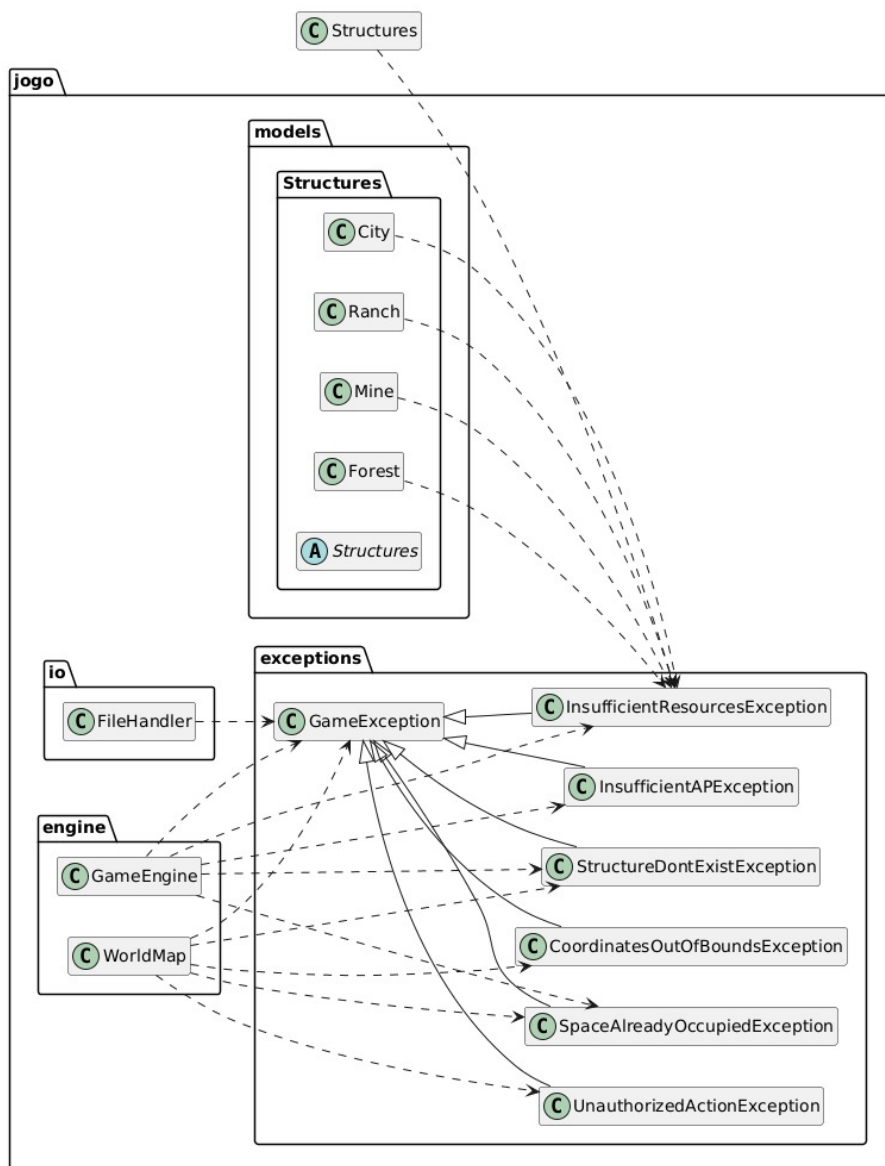
UML de jogo.io



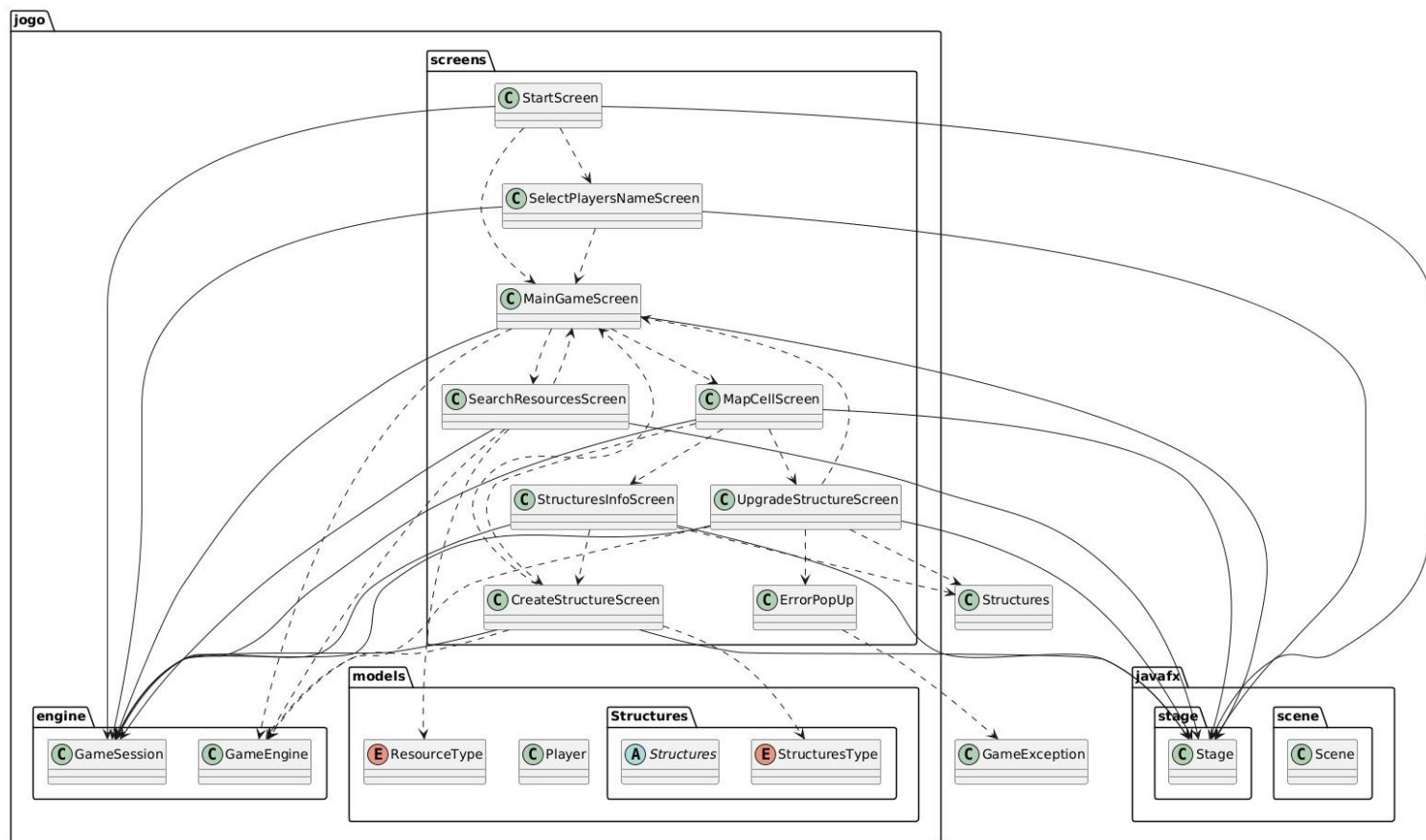
UML de jogo.exceptions



UML de jogo



UML de Relações de `jogo.models.structures` e `jogo.exceptions`



UML de jogo.screens

## Arquitetura da Aplicação

A aplicação foi organizada de forma modular, separando e respeitando as diferentes responsabilidades em packages distintos. Esta organização permitiu uma melhor manutenção do código, facilita a leitura do projeto e respeita os princípios da POO.

### Modelo de Domínio

O modelo de domínio encontra-se principalmente no package `jogo.models` e no package `jogo.models.Structures`.

O package `jogo.models` contém a classe `Player`, responsável por representar os jogadores, e o enum `ResourceType`, que define os tipos de recursos disponíveis no jogo. O package `jogo.models.Structures` contém a classe abstrata `Structures` e as suas sub-classes concretas: `Forest`, `Mine`, `Ranch` e `City`. Estas classes representam as estruturas que podem ser construídas no mapa do jogo. Cada estrutura possui as suas próprias características.

### Controlo da Aplicação

O controlo principal da aplicação encontra-se no package `jogo.engine`.

A classe `GameSession` é responsável por controlar a sessão atual do jogo, incluindo os jogadores, o mapa, o dia atual e os eventos. Esta classe gere a passagem de turnos, a progressão dos dias e a verificação de fim de jogo.

A classe `GameEngine` contém a lógica das ações principais do jogo. Esta classe trata da



criação de estruturas, da melhoria de estruturas, da procura de recursos e na obtenção de informações.

A classe WorldMap representa o mapa do jogo e controla o posicionamento, a consulta e as validações das estruturas na grelha.

A classe Events gere os eventos aleatórios que podem ocorrer em cada dia.

## Interface Gráfica

A interface gráfica foi desenvolvida em JavaFX e encontra-se no package jogo.screens.

Este package contém as classes responsáveis pelos diferentes ecrãs da aplicação, como o ecrã inicial e o ecrã principal do jogo.

As classes da interface gráfica comunicam com a lógica do jogo através da GameSession e do GameEngine. Assim, a Interface é responsável por recolher as ações do jogador e apresentar os resultados.

## Funcionalidades Implementadas

Foram implementadas várias funcionalidades no jogo.

A primeira funcionalidade é a criação de uma nova sessão de jogo. O utilizador pode iniciar um novo jogo, com os nomes escolhidos, com o mapa vazio e no primeiro dia de jogo.

Foi implementado um sistema de turnos. Cada jogador tem uma quantidade de ações limitadas em cada turno, com base no AP do jogador. Quando o jogador termina o turno, o controlo do jogo passa para o adversário. Após ambos terminarem o turno, o dia avança.

Também foi implementado um sistema de recursos. Os jogadores possuem madeira, pedra, comida e pontos de ação. Estes recursos são necessários para construir e melhorar estruturas e para mantê-las “vivas”.

A funcionalidade de construção permite ao jogador escolher uma posição do mapa e construir uma estrutura, desde que possua os recursos e o AP necessários. As estruturas disponíveis são Floresta, Mina, Rancho e Cidade.

Foi implementada a funcionalidade de melhoria de estruturas. O jogador pode melhorar uma estrutura que lhe pertença, aumentando o seu nível e alterando os seus valores de produção, manutenção e pontuação.

Existe também a funcionalidade de procura de recursos. Esta permite ao jogador gastar AP para obter uma quantidade aleatória de um recurso específico.

O jogo inclui eventos aleatórios diários, estes eventos podem alterar a produção de recursos ou a pontuação obtida através das estruturas, o que torna a jogabilidade mais dinâmica.

Foi implementado um sistema de manutenção das estruturas. Ao final de cada dia, as estruturas consomem recursos. Caso o jogador não tenha recursos suficientes, a

estrutura pode perder um nível ou ser destruída.

A aplicação também permite guardar e carregar o jogo. O estado do jogo é guardado num ficheiro CSV, contendo os jogadores, os seus recursos, a pontuação, o dia atual e as estruturas existentes no mapa.

Por fim, foi implementada a verificação do fim de jogo. Quando o número máximo de dias é ultrapassado, o jogo termina e é determinado o vencedor com base na pontuação dos jogadores.

## Aplicação dos Conceitos de POO

Durante o desenvolvimento do projeto, foram aplicados diversos conceitos fundamentais de POO, tais como o encapsulamento, a herança e o polimorfismo.

### Encapsulamento

O encapsulamento foi aplicado através da utilização de atributos privados ou protegidos e métodos públicos de acesso e modificação controlada.

Como por exemplo, na classe Player, os atributos como o nome, a pontuação, o AP base e o inventário não são manipulados diretamente a partir do exterior. São utilizados métodos como getName, addResource, removeResource e getResourceQuantity.

Esta abordagem permite proteger o estado interno dos objetos e garantir que as alterações são feitas de maneira controlada.

### Herança

A herança foi aplicada na hierarquia das estruturas e das exceções.

A classe Abstract Structures representa a estrutura base de todas as construções do jogo. Esta classe define os atributos e os métodos comuns, como o nível, o dono e métodos abstratos relacionados com a geração de recursos.

As classes Forest, Mine, Ranch, City herdam de Structures. Cada uma destas classes representa um tipo específico de estrutura e implementa os métodos abstratos de acordo com as suas próprias regras.

A herança também foi aplicada no sistema de exceções. A classe GameException funciona como a exceção base do projeto. As exceções específicas, como a InsufficientAPException e UnauthorizedActionException, herdam de GameException.

### Polimorfismo

O polimorfismo foi aplicado através da utilização da classe abstrata Structures para representar qualquer tipo de estrutura concreta.

No mapa, as estruturas são guardadas numa matriz do tipo Structures. Isto permite que uma posição do mapa contenha uma Forest, uma Mine, um Ranch ou uma City, tratando todas como objetos do tipo base Structures.

Apesar de todas as estruturas serem tratadas de forma genérica, cada uma pode ter

comportamentos diferentes. Por exemplo, ao chamar o método `generateResource`, cada estrutura executa a sua própria implementação. A Floresta produz madeira, a Mina produz pedra, o Rancho produz comida e a Cidade não produz recursos materiais. Este mecanismo permite tornar o código mais flexível e reutilizável, o que reduz a repetição e facilita a implementação de novas estruturas.

#### Tratamento de Exceções

O projeto utiliza exceções personalizadas para tratar situações de erro e impedir ações inválidas durante o jogo.

A exceção base é `GameException`, que representa erros genéricos relacionados com a lógica do jogo. A partir desta classe foram criadas várias exceções específicas.

A exceção `InsufficientAPException` é utilizada quando o jogador tenta realizar uma ação sem ter o AP necessário.

A exceção `InsufficientResourcesException` é utilizada quando o jogador não possui recursos suficientes para construir ou melhorar uma estrutura.

A exceção `SpaceAlreadyOccupiedException` é lançada quando o jogador tenta construir uma estrutura numa posição do mapa que já se encontra ocupada.

A exceção `StructureDontExistException` é utilizada quando o jogador tenta interagir com uma posição onde não existe estrutura alguma.

A exceção `UnauthorizedActionException` é utilizada quando o jogador tenta interagir com uma estrutura que pertence ao oponente.

A exceção `CoordinatesOutOfBoundsException` é utilizada quando são indicadas coordenadas fora dos limites do mapa.

## Testes Unitários

Durante o desenvolvimento do projeto foram realizados testes unitários para verificar o correto funcionamento das principais classes da aplicação. Estes testes tiveram como objetivo garantir que a lógica do jogo funcionava corretamente, tanto em situações normais como em situações de erro.

Na classe `Player`, foram testados métodos relacionados com a criação de jogadores, a validação do nome, a pontuação inicial, os recursos iniciais, a adição e remoção de recursos e a reposição dos pontos de ação. Também foram considerados cenários inválidos, como a tentativa de criar um jogador com nome vazio ou nulo.

Na classe `WorldMap`, foram testados métodos como `addStructure`, `getStructure`, `isNotOccupied`, `validateCoordinates` e `canInteract`. Estes testes permitiram verificar se as estruturas eram corretamente colocadas no mapa, se era possível consultar uma estrutura existente e se eram lançadas exceções quando as coordenadas estavam fora dos limites ou quando uma posição já estava ocupada.

Na classe GameEngine, foram testadas funcionalidades como a criação de estruturas, a melhoria de estruturas e a procura de recursos. Foram considerados cenários em que o jogador possuía recursos e pontos de ação suficientes, bem como cenários inválidos em que o jogador não tinha recursos ou AP suficientes para realizar a ação.

Nas classes das estruturas, como Forest, Mine, Ranch e City, foram testados métodos relacionados com a produção de recursos, o consumo de recursos, a melhoria de nível e a alteração dos valores de produção e manutenção. Também foi testado se cada estrutura tinha o comportamento esperado de acordo com o seu tipo.

Para além disso, foram testadas situações relacionadas com o tratamento de exceções, como a tentativa de construir numa posição ocupada, interagir com uma estrutura inexistente, utilizar coordenadas inválidas ou realizar ações sem recursos suficientes.

Desta forma, os testes unitários contribuíram para aumentar a fiabilidade do projeto e ajudaram a identificar e corrigir erros durante o desenvolvimento.

## Dificuldades Estruturas

Durante o desenvolvimento do projeto surgiram algumas dificuldades.

Uma das principais dificuldades foi a organização do projeto em diferentes packages e a separação correta entre a lógica do jogo e a interface gráfica. Inicialmente, algumas responsabilidades estavam menos claras, mas esta dificuldade foi ultrapassada através da divisão do projeto em packages como jogo.engine, jogo.models, jogo.models.Structures, jogo.screens, jogo.io e jogo.exceptions.

Outra dificuldade esteve relacionada com a gestão dos turnos e da progressão dos dias. Foi necessário garantir que os jogadores alternavam corretamente, que os pontos de ação eram repostos no momento certo e que a produção e o consumo de recursos aconteciam apenas no final de cada dia.

A implementação do sistema de gravação e carregamento do jogo também exigiu atenção. Foi necessário garantir que os dados dos jogadores, os recursos, a pontuação, o dia atual e as estruturas existentes no mapa eram guardadas e carregadas corretamente. Esta dificuldade foi resolvida através da classe FileHandler.

## Conclusão

Com a realização deste projeto foi possível desenvolver um jogo de estratégia por turnos funcional, utilizando Java e JavaFX.

O jogo permite a criação de dois jogadores, a gestão de turnos, a construção e melhoria de estruturas, a produção e consumo de recursos, a ocorrência de eventos aleatórios, a gravação e carregamento do progresso e a determinação do vencedor no final da partida.

Durante o desenvolvimento foram aplicados vários conceitos de Programação Orientada a Objetos, como o encapsulamento, a herança e o polimorfismo. A utilização da classe abstrata Structures permitiu organizar melhor o código e evitar repetição, uma vez que as estruturas concretas partilham características comuns, mas possuem comportamentos próprios.

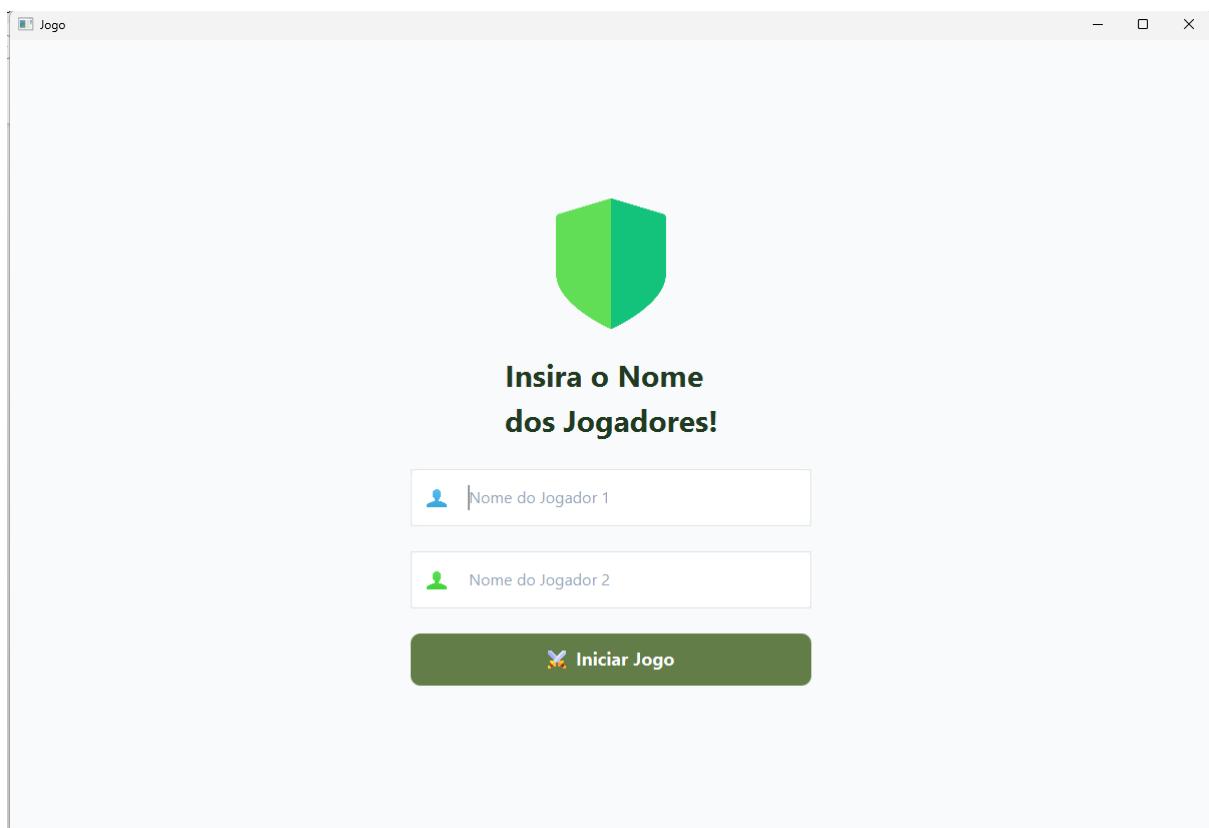
A divisão do projeto em packages também contribuiu para uma melhor organização da aplicação, tornando mais clara a separação entre o modelo de domínio, o controlo da aplicação, a interface gráfica, o tratamento de exceções e a persistência de dados.

Apesar das dificuldades encontradas, o projeto permitiu consolidar conhecimentos de Java, JavaFX, UML, testes unitários, tratamento de exceções e Programação Orientada a Objetos. O resultado é uma aplicação organizada, funcional e adequada aos objetivos propostos.

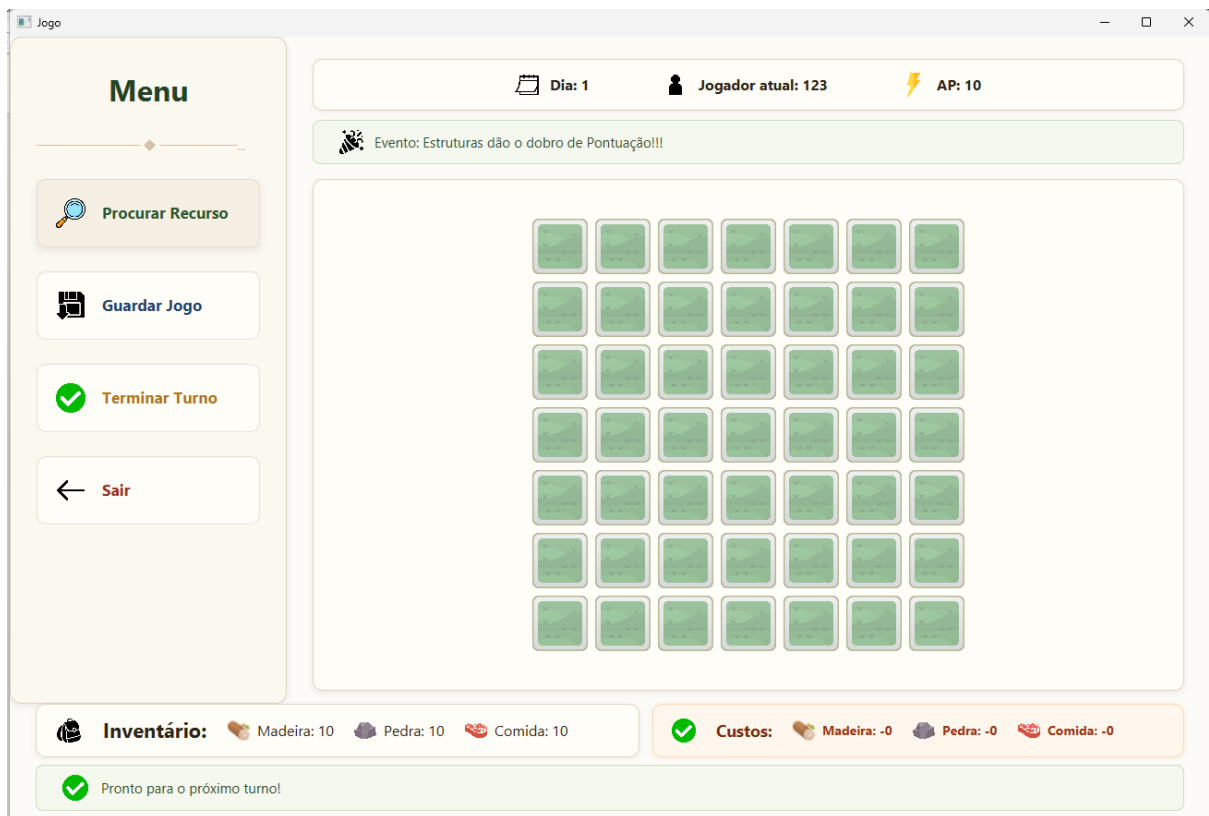
## Captura de Ecrã



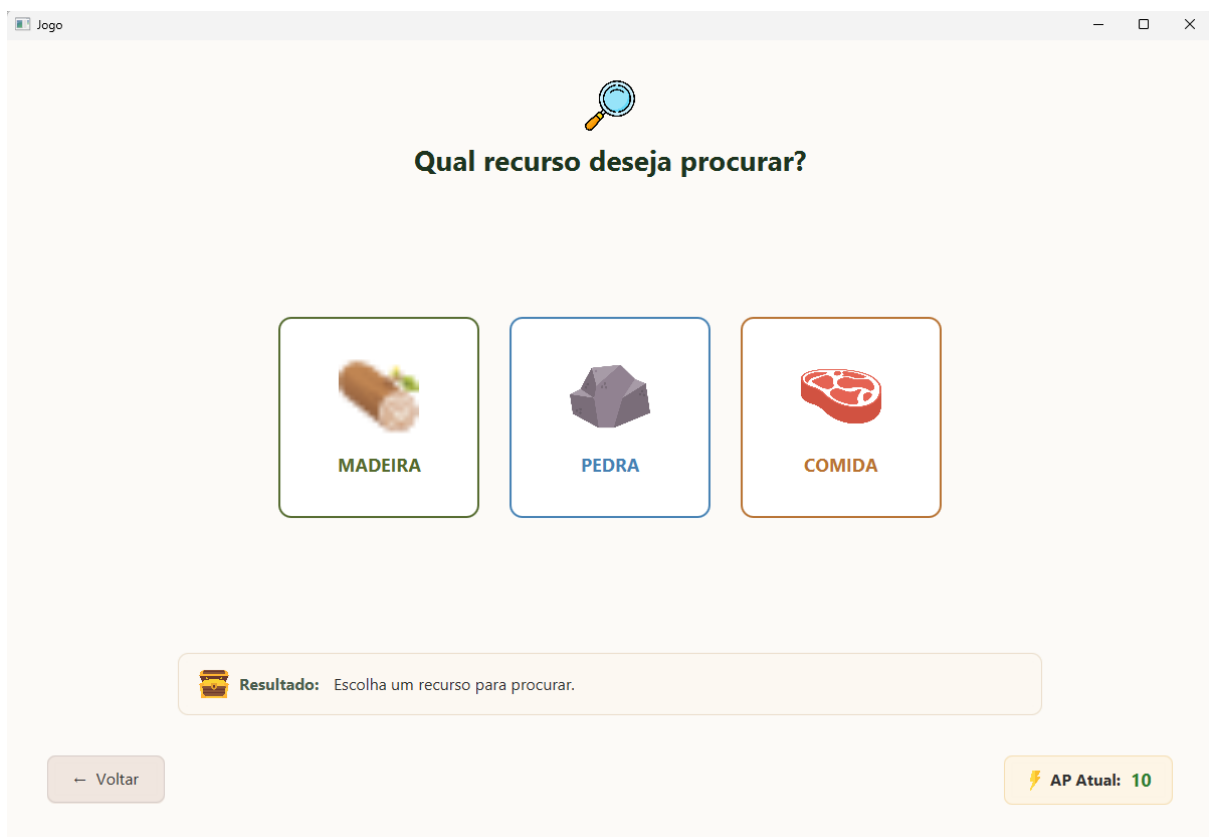
*Tela Inicial*



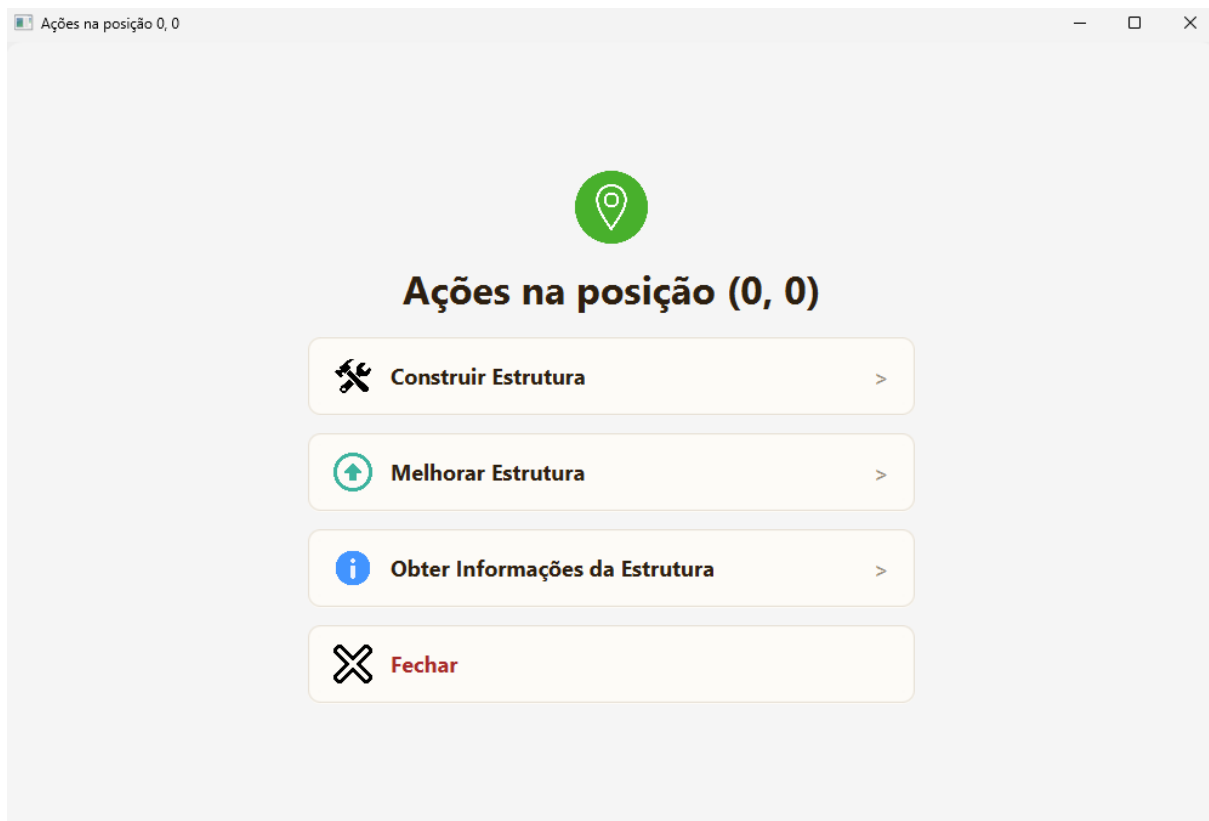
*Tela de escolher o nome dos jogadores*



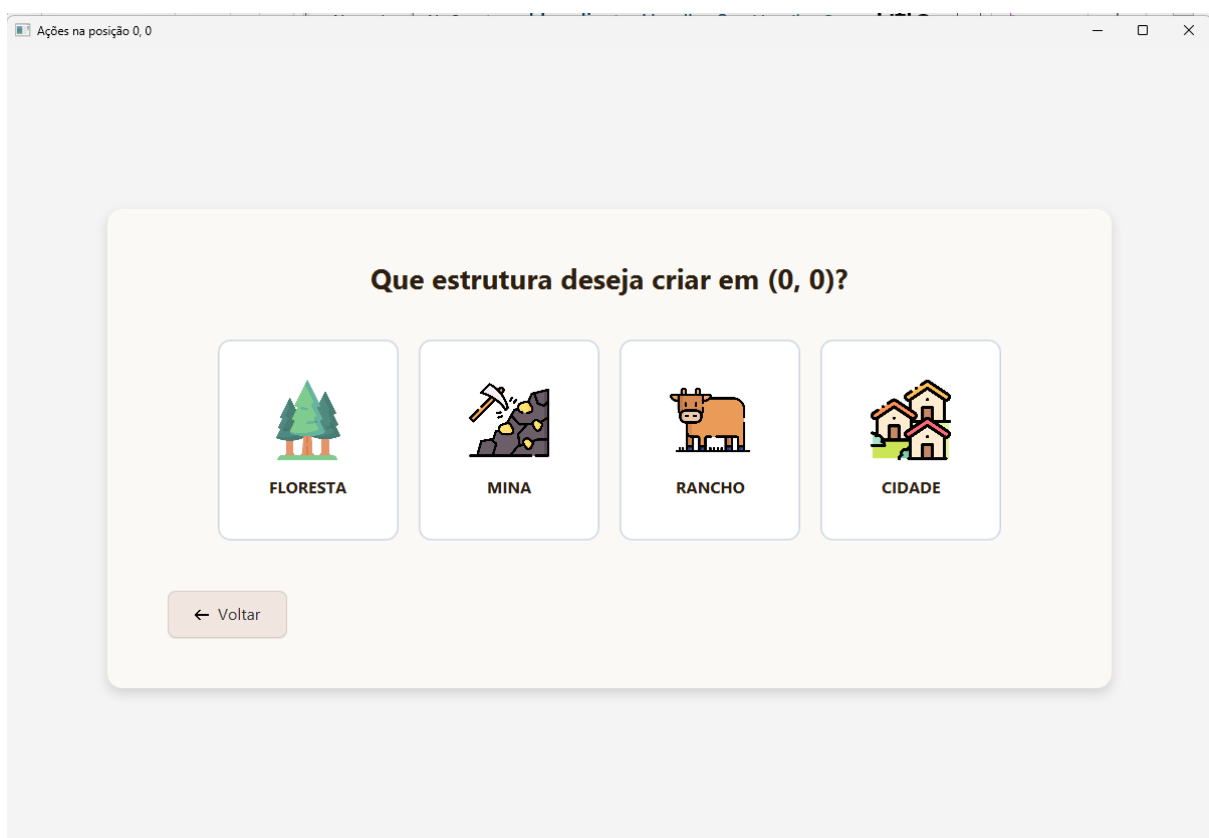
Tela principal do jogo



Tela de procura de recursos

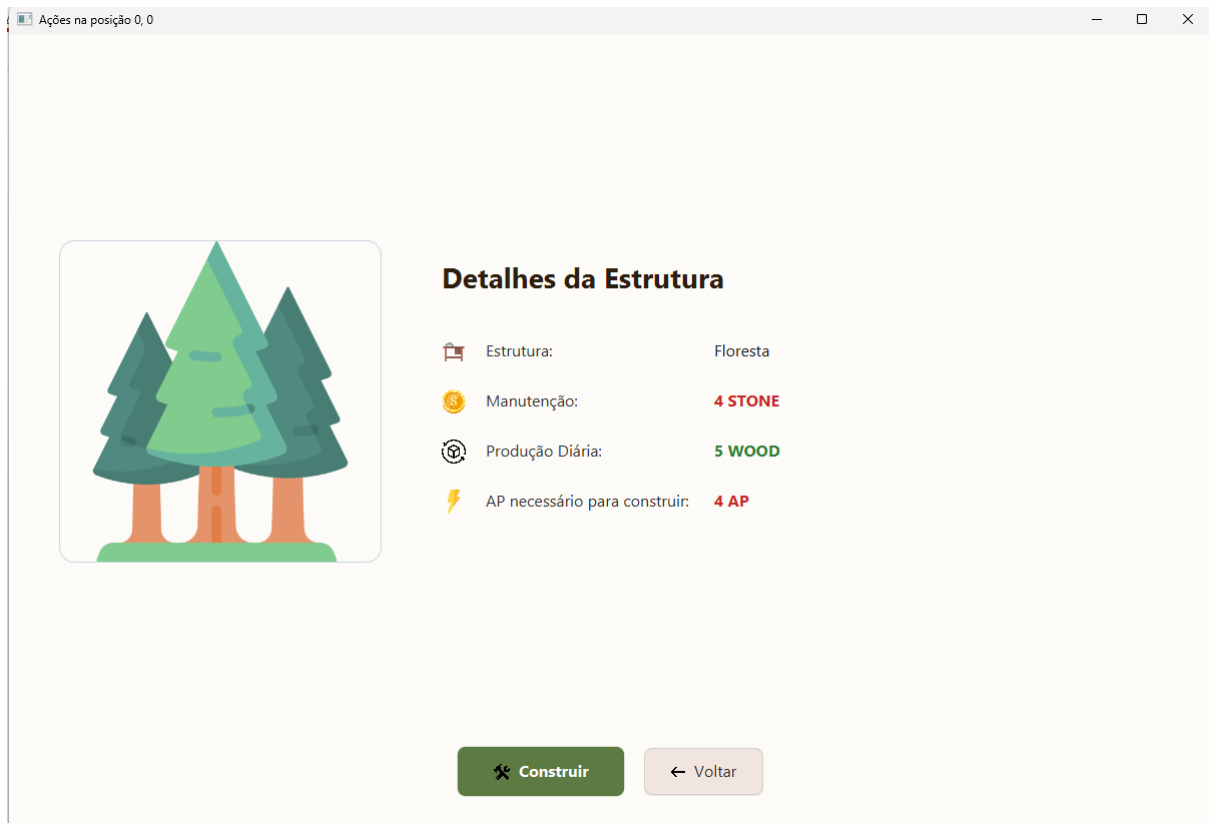


Tela de Ações no mapa

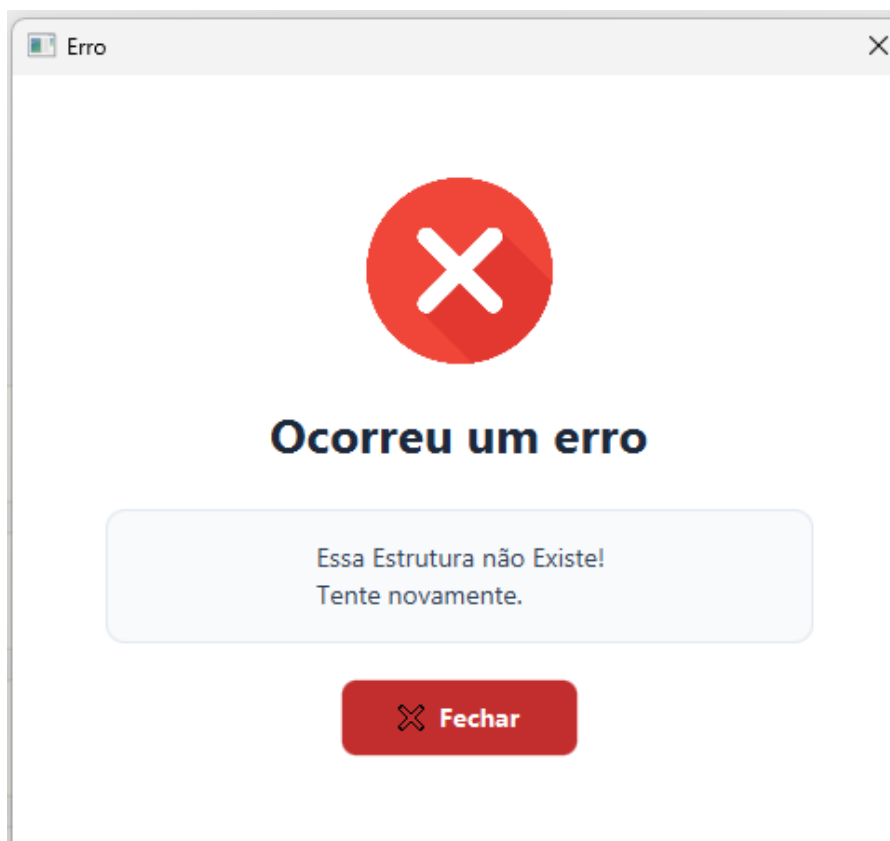


Tela de Seleção de Estrutura

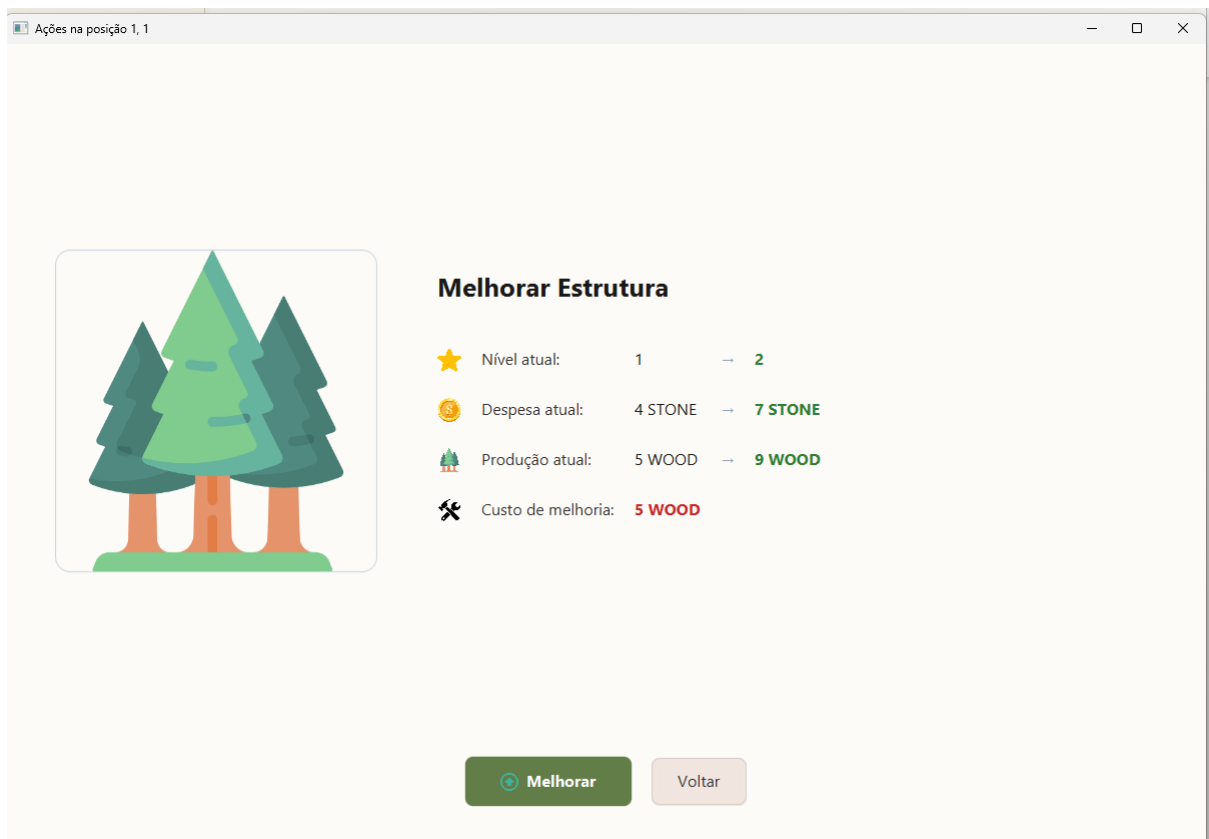




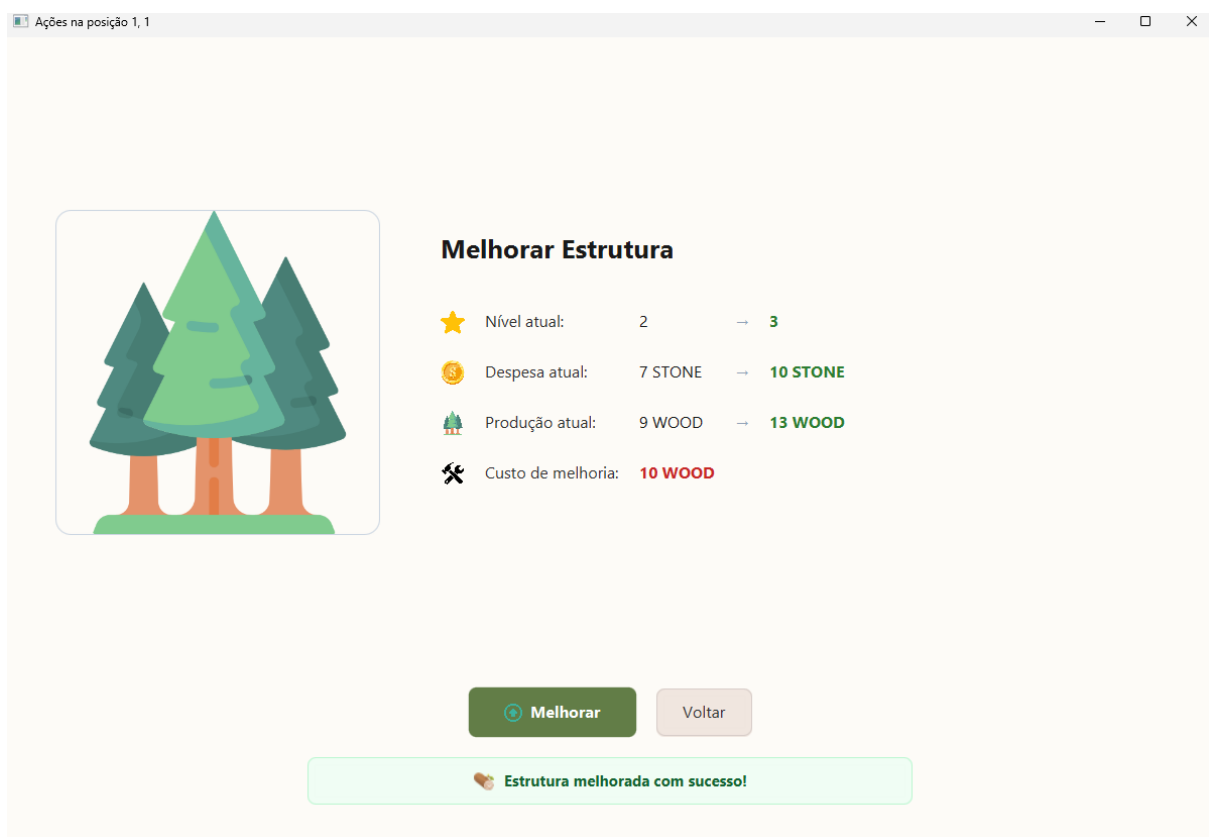
Tela de Construção de Estrutura



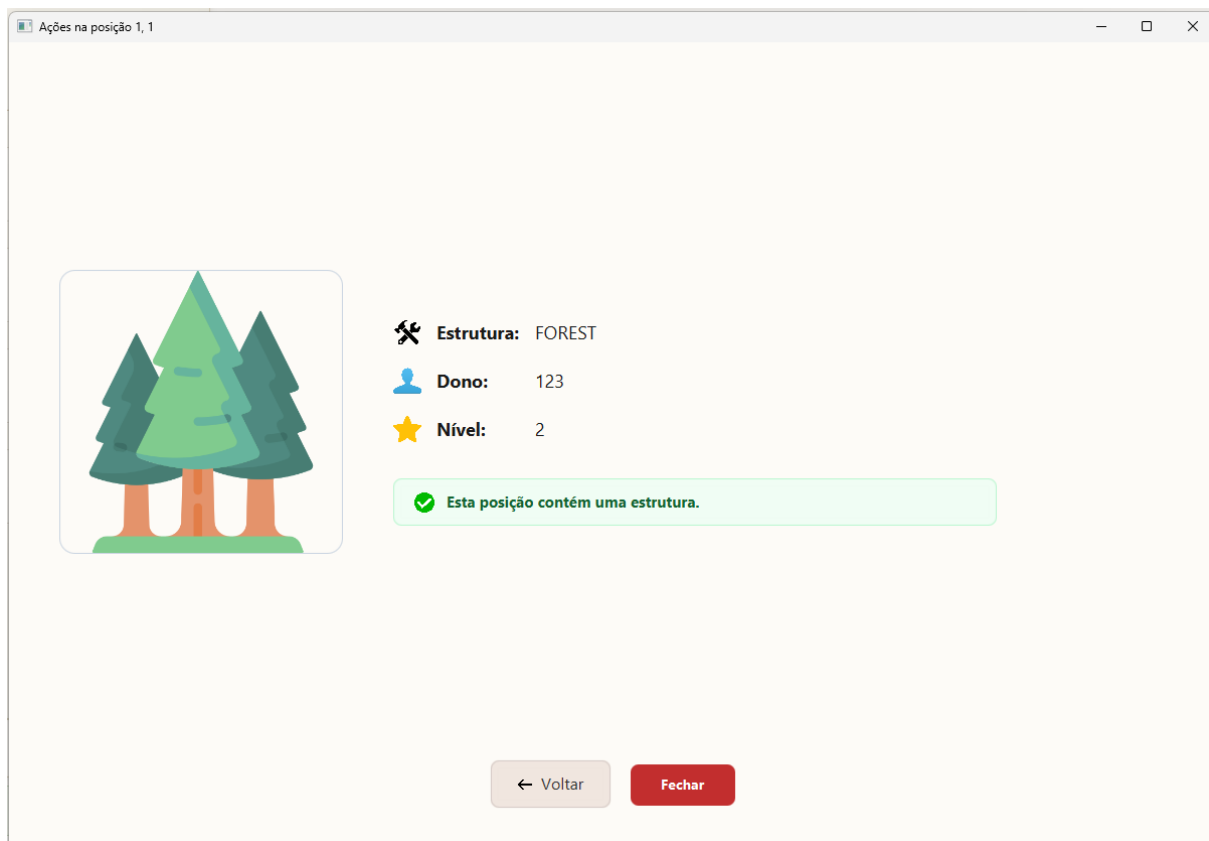
Exemplo de Tela de Erro



Tela de Melhoria de Estrutura



Tela Após melhorar a Estrutura



Tela de Obter informações do mapa



Tela final de vencedor